Dylan Flores
October 25 2026
CS 240 - 01

Assignment 5 Assembler and Disassembler Project

Assembler and disassembler in Java tested on my FizzBuzz program

All code github.com/dflo042/cs/tree/main/assembler-project

Part 1 Assembling the four instructions by hand

addi is I format. The fields are opcode 6 bits then rs 5 bits then rt 5 bits then a 16 bit immediate. add and sub are R format. The fields are opcode 6 then rs 5 then rt 5 then rd 5 then shamt 5 then funct 6. The opcode for every R format instruction is 0 and the funct field says which one it is. The register numbers I needed are $zero 0 and $t0 8 and $t1 9 and $t2 10 and $t3 11.

addi $t0, $zero, 3 is I format

Field	Value	Bits
opcode addi	8	`001000`
rs $zero	0	`00000`
rt $t0	8	`01000`
immediate	3	`0000000000000011`

Put together in order that is 0010 0000 0000 1000 0000 0000 0000 0011 which is 0x20080003

addi $t1, $zero, 5 is I format

Field	Value	Bits
opcode addi	8	`001000`
rs $zero	0	`00000`
rt $t1	9	`01001`
immediate	5	`0000000000000101`

Put together in order that is 0010 0000 0000 1001 0000 0000 0000 0101 which is 0x20090005

add $t2, $t0, $t1 is R format

Field	Value	Bits
opcode	0	`000000`
rs $t0	8	`01000`
rt $t1	9	`01001`
rd $t2	10	`01010`
shamt	0	`00000`
funct add	32	`100000`

Put together in order that is 0000 0001 0000 1001 0101 0000 0010 0000 which is 0x01095020

sub $t3, $t1, $t0 is R format

Field	Value	Bits
opcode	0	`000000`
rs $t1	9	`01001`
rt $t0	8	`01000`
rd $t3	11	`01011`
shamt	0	`00000`
funct sub	34	`100010`

Put together in order that is 0000 0001 0010 1000 0101 1000 0010 0010 which is 0x01285822

To check my work I put the same four lines in a file and dumped the text segment from MARS and it gave 20080003 20090005 01095020 01285822 which matches. My assembler gives the same four words and that is one of the tests.

Part 2 Machine code as bytes

Each 32 bit word splits into four 8 bit bytes. I list them with the most significant byte first which is the order my .mhc-bin file writes them so the opcode is in the first byte.

Instruction	Byte 0	Byte 1	Byte 2	Byte 3	Hex
addi $t0, $zero, 3	`00100000`	`00001000`	`00000000`	`00000011`	`20 08 00 03`
addi $t1, $zero, 5	`00100000`	`00001001`	`00000000`	`00000101`	`20 09 00 05`
add $t2, $t0, $t1	`00000001`	`00001001`	`01010000`	`00100000`	`01 09 50 20`
sub $t3, $t1, $t0	`00000001`	`00101000`	`01011000`	`00100010`	`01 28 58 22`

How the two output formats differ

In the .mhc-ascii file every bit is a whole character. The character 0 is stored as the byte 0x30 and the character 1 is stored as 0x31 so one instruction takes 32 bytes on disk plus a newline. You can open it in any text editor and read the bits but nothing can run it as is. In the .mhc-bin file the 32 bits are packed into 4 real bytes. The first byte of addi $t0, $zero, 3 is 0x20 which is 00100000 and that is the opcode plus the top two bits of rs. The bin file is what a machine would actually load and you need a hex viewer or a program to look at it.

My FizzBuzz has 45 instructions and 19 data bytes and my files add a 5 word header so each one holds 55 words. fizzbuzz.mhc-bin is 220 bytes and fizzbuzz.mhc-ascii is 1817 bytes which is a little over eight times bigger. The information inside is identical and that is why the disassembler can read either file and gives back the same program. Here is the start of each file. The ascii one is the first three lines and the bin one is the first 24 bytes shown in hex.

```
01001101010010000100001100110001     the letters MHC1
00000000010000000000000000000000     text starts at 0x00400000
00000000000000000000000000101101     45 instruction words

4d 48 43 31  00 40 00 00  00 00 00 2d  10 01 00 00  00 00 00 13  24 09 00 01
```

Part 3 The assembler

Assembler.java is one file with no libraries. Run javac Assembler.java and then java Assembler fizzbuzz.asm and it writes fizzbuzz.mhc-ascii and fizzbuzz.mhc-bin next to the source. It makes two passes. The first pass strips comments and splits the file into .text and .data and records the address of every label and expands each pseudoinstruction into real instructions so the addresses are right. The second pass encodes every instruction into its 32 bit word now that all the labels are known. Data is kept as a plain list of bytes.

Supported features

- Directives .text .data .globl .asciiz .ascii .word .byte .space .align
- Real instructions add addu sub subu and or xor nor slt sltu sll srl sra jr jalr mult multu div divu mfhi mflo syscall break addi addiu slti sltiu andi ori xori lui lw sw lb lbu lh lhu sb sh beq bne j jal
- Pseudoinstructions li la move rem div nop b beqz bnez blt bgt ble bge and lw or sw with a label
- Labels on their own line or in front of an instruction or data. Comments with #. Registers by name like $t0 or by number like $8

The pseudoinstructions expand exactly the way MARS 4.5 expands them. I read the expansions out of the PseudoOps.txt file that ships inside MARS so my machine code matches a MARS dump word for word. These are the four that FizzBuzz uses.

Pseudo	What it becomes
`li $t1, 1`	`addiu $t1, $zero, 1 (ori for 32768 to 65535 and lui plus ori for anything bigger)`
`la $a0, label`	`lui $at, 4097 then ori $a0, $at, offset`
`move $a0, $t1`	`addu $a0, $zero, $t1`
`rem $t0, $t1, $t8`	`bne $t8, $zero, 1 then break then div $t1, $t8 then mfhi $t0`

File layout and addresses and byte order

Both files hold the same list of 32 bit words. The text segment starts at 0x00400000 and the data segment starts at 0x10010000 which are the MARS defaults so la gives the same lui and ori that MARS gives. The .mhc-bin file writes every word most significant byte first so the opcode is the first byte. The .mhc-ascii file writes every word as 32 zero and one characters on its own line with a blank line between the header and the text and the data. The words are laid out like this.

Word	Meaning
0	the letters MHC1 as one word (0x4D484331) so the disassembler knows the file is mine
1	text start address 0x00400000
2	number of instruction words
3	data start address 0x10010000
4	number of data bytes
5 and on	the instruction words and then the data bytes four per word in address order padded with zeros

The header is how instructions are told apart from data. Word 2 says how many words after the header are instructions and word 4 says how many bytes after that are data so nothing has to be guessed. Data bytes go in address order. A .word value is stored little endian the way MARS stores it so .word 7 becomes the bytes 07 00 00 00 and a string is stored one character per byte with a zero on the end for .asciiz. In FizzBuzz the three strings are packed back to back so Fizz is at 0x10010000 and Buzz is at 0x10010005 and FizzBuzz is at 0x1001000a and that is exactly what MARS does too.

Errors

Anything the assembler does not understand stops it with a message that says the line and the problem. These are the real messages from my error tests.

```
error line 3 unsupported instruction mul in mul $t1, $t0, $t0
error line 3 unknown register $t9x
error line 3 label nowhere is not defined anywhere
error line 2 immediate 40000 does not fit in 16 signed bits
error line 2 add needs 3 operands but got 2
error line 2 .asciiz needs a string in double quotes
```

Part 4 The disassembler

Disassembler.java reads either file. It looks at the contents not the file name. If the first four bytes spell MHC1 it is the binary file and the words are read four bytes at a time. Otherwise every line is checked to be exactly 32 zeros and ones and parsed as a word. Then it checks the header against the file length and decodes every instruction word. Branch and jump targets get labels named L1 L2 and so on and the first instruction is labeled main. The data bytes come back as .byte lines so the exact bytes are kept. Run it with java Disassembler fizzbuzz.mhc-bin out.asm.

Comments and the original label names and the pseudoinstruction spellings are gone which the assignment says is fine. This is the start of fizzbuzz_rebuilt.asm. The whole file is on GitHub.

```
.data
.byte 70, 105, 122, 122, 0, 66, 117, 122
.byte 122, 0, 70, 105, 122, 122, 66, 117
.byte 122, 122, 0

.text
main:
addiu $t1, $zero, 1
addiu $t2, $zero, 101
addiu $t3, $zero, 3
addiu $t5, $zero, 5
addiu $t8, $zero, 15
L1:
bne $t8, $zero, L2
break
L2:
div $t1, $t8
mfhi $t0
beq $t0, $zero, L5
...
```

You can see the li lines came back as addiu and the first rem came back as its four real instructions with a label on the instruction after the break. Assembling fizzbuzz_rebuilt.asm gives files that are byte for byte identical to fizzbuzz.mhc-bin and fizzbuzz.mhc-ascii and running it in MARS prints the same 100 lines as the original.

Tests

tests/run_tests.sh compiles both programs and runs 28 checks. If Mars.jar is in the folder it also cross checks against MARS. Every check passed and the full output is saved in tests/test_log.txt on GitHub.

Test	What it checks	Result
1	fizzbuzz.asm assembles and both output files match the ones in the repo	pass
2	the ascii file is the bin file written as 0 and 1 characters word for word	pass
3	the bin file and the ascii file disassemble to the same program and it matches fizzbuzz_rebuilt.asm	pass
4	the rebuilt program assembles again and the bytes are identical to the originals	pass
5	the four Part 1 instructions assemble to the words I got by hand	pass
6	six bad .asm files are rejected. unknown instruction and bad register and missing label and immediate too big and wrong operand count and bad string	pass
7	six bad machine code files are rejected. cut off bin file and header count wrong and 31 character line and a 2 in a line and plain text and an unknown opcode	pass
8	my 45 text words equal the MARS dump of fizzbuzz.asm and the original and rebuilt programs print the same thing in MARS and that output is the correct FizzBuzz 1 to 100	pass

The end of the test log

```
== 7 bad machine code input gives an error
PASS cutoff.mhc-bin rejected. error tests/cutoff.mhc-bin has 25 bytes which is not a
     multiple of 4. The file is cut off or damaged
PASS wrong_count.mhc-bin rejected. error tests/wrong_count.mhc-bin header says 2 text
     words and 0 data bytes but the file holds 1 words after the header
PASS short_line.mhc-ascii rejected. error tests/short_line.mhc-ascii line 7 has 31
     characters. Every line must have exactly 32 zeros and ones
PASS bad_char.mhc-ascii rejected. error tests/bad_char.mhc-ascii line 7 has a
     character that is not 0 or 1
PASS not_mhc.mhc-ascii rejected. error tests/not_mhc.mhc-ascii line 1 has 30
     characters. Every line must have exactly 32 zeros and ones
PASS unknown_opcode.mhc-ascii rejected. error unsupported opcode 0x3f in word
     0xfc000000 at 0x00400000
== 8 cross check with MARS
PASS text words equal the MARS dump
PASS original and rebuilt print the same thing in MARS
PASS rebuilt output is the correct FizzBuzz 1 to 100

28 passed 0 failed
```

For the MARS check I ran java -jar Mars.jar nc a dump .text HexText out.txt fizzbuzz.asm which prints one hex word per instruction and compared it to the 45 words in my bin file. The first few are 24090001 240a0065 240b0003 and the last is 0000000c for the exit syscall. Then I ran both fizzbuzz.asm and the rebuilt file in MARS and compared all 100 lines.

Sources and acknowledgments

- ZyBooks Chapter 4 for the R and I and J formats and the field sizes and the opcode and funct values.
- The PseudoOps.txt file inside MARS 4.5 for how li la move and rem expand and the MARS command line dump for checking my words. MARS is by Pete Sanderson and Ken Vollmar (dpetersanderson.github.io).
- fizzbuzz.asm is my own program from Assignment 3. No code was copied from anywhere.
- Testing was done from the command line with the shell script in the tests folder and OpenJDK 21.